

# PLANDEX\_PORTING\_CHECKLIST

## Plandex Porting Checklist for HelixCode

### Phase 1: LLM Provider Integration (2-3 weeks)

#### 1.1 Provider Framework Setup

☐

Create internal/llm/provider/ package structure

☐

interface.go: Unified Provider interface

☐

openai.go: OpenAI direct API

☐

anthropic.go: Anthropic direct API

☐

google.go: Google AI Studio + Vertex

☐

azure.go: Azure OpenAI

☐

aws.go: AWS Bedrock

☐

openrouter.go: OpenRouter aggregator

☐

deepseek.go: DeepSeek

☐

perplexity.go: Perplexity

☐

ollama.go: Ollama local

☐

custom.go: Custom OpenAI-compatible

#### 1.2 Credential Management

☐

Environment variable discovery system

☐

- ☐ `credentials/env_loader.go`
- ☐ Support for:
  - ☐ Simple API keys (OpenAI, Anthropic, etc.)
  - ☐ OAuth tokens (Anthropic Claude Max)
  - ☐ AWS credential chain + profile
  - ☐ Google service account (file/JSON/base64)
  - ☐ Azure (API key + base URL)
- ☐ Lazy loading (credentials loaded only when provider used)
- ☐ Validation and error messages for missing credentials

### 1.3 Provider Fallback Chain

- ☐ `internal/llm/router/fallback.go`
  - ☐ Fallback configuration (priority order)
  - ☐ Automatic retry with next provider on error
  - ☐ Provider availability checking
  - ☐ Cost/speed preference tuning
- ☐ Error categorization (retryable vs. fatal)
- ☐ Circuit breaker pattern for failing providers

### 1.4 LiteLLM Proxy Setup (Optional but Recommended)

- ☐ Python FastAPI proxy server
  - ☐ Port 4000 (standard)
  - ☐ Health check endpoint `/health`
  - ☐

- ☐ /v1/chat/completions passthrough
- ☐ OAuth header handling for Anthropic
- ☐ Streaming response support
- ☐ Error handling and logging
- ☐ Docker integration
  - ☐ Python + Go base image
  - ☐ Environment variable passing
  - ☐ Port exposure

## 1.5 Testing

- ☐ Unit tests for each provider
  - ☐ Mock provider for testing
  - ☐ Integration tests with fallback chain
  - ☐ Credential loading tests (with env var fixtures)
- 

# Phase 2: Model Configuration & Role System (2-3 weeks)

## 2.1 Model Metadata Schema

- ☐ Database migrations for model tables
  - ☐ models table (model definitions)
  - ☐ model\_variants table (variants with overrides)
  - ☐ model\_providers table (provider associations)
  - ☐ model\_packs table (role → model mappings)

- ☐
  - ☐ model\_pack\_configs table (pack role configs)
- ☐ Data types:
  - ☐ ModelTag, ModelId, ModelName, VariantTag
  - ☐ MaxTokens, MaxOutputTokens, ReservedOutputTokens
  - ☐ DefaultMaxConvoTokens
  - ☐ PreferredOutputFormat (JSON vs XML)
  - ☐ ModelCompatibility flags (image support, etc.)
  - ☐ SupportsCacheControl
  - ☐ SystemPromptDisabled, RoleParamsDisabled, StopDisabled

## 2.2 Model Roles System

- ☐ Define 9 roles in code:
  - ☐ planner: Main planning + prompt replies
  - ☐ architect: High-level planning + context selection (optional)
  - ☐ coder: Code implementation with strict formatting (optional)
  - ☐ builder: Convert plans to file diffs
  - ☐ whole-file-builder: File rewrite fallback (optional)
  - ☐ summarizer: Conversation summarization
  - ☐ auto-continue: Determine plan completion
  - ☐ names: Auto-name plans and context
  - ☐ commit-messages: Generate git commit messages
- ☐ Role descriptions and configuration



Role fallback chain (coder → builder → planner, etc.)

## 2.3 Built-in Model Definitions



Create model definitions file with 25+ models:



OpenAI: o3, o4-mini, gpt-4.1, gpt-4o, gpt-4o-mini, gpt-4-turbo



Anthropic: Claude 3.5 Sonnet, 3 Opus, 3 Haiku, Extended context



Google: Gemini 2.0 Flash, 1.5 Pro/Flash/Exp



DeepSeek: R1, V3



Perplexity: Sonar



Local/OSS: Mistral, Qwen, Llama



Variants for reasoning models (high/medium/low effort)



Large context fallbacks configuration

## 2.4 Built-in Model Packs (16 total)



Create model pack definitions:



General: DailyDriver (default), Strong, Cheap, OSS, Reasoning



Provider-specific: AnthropicOnly, OpenAIOnly, GoogleOnly



Planner-specific: OpusPlanner, O3Planner, R1Planner, GeminiPlanner, PerplexityPlanner



Local: OllamaExperimental, OllamaAdaptiveOSS, OllamaAdaptiveDaily



Pack role assignments for each



Default fallbacks for unassigned roles

## 2.5 Model Resolution Engine



`internal/llm/resolver/` package



`resolve_model.go`: Plan + role → Model



`get_provider.go`: Model → Provider instance



`validate_availability.go`: Check provider credentials



`fallback.go`: Use alternative if unavailable



Runtime checking of:



Model support for required capabilities (image, JSON, etc.)



Provider authentication availability



Context size constraints (MaxTokens)

## 2.6 Handlers



`internal/server/handlers/models.go`



GET `/api/v1/models` - List available models



GET `/api/v1/models/:id` - Get model details



GET `/api/v1/model-packs` - List packs



GET `/api/v1/model-packs/:name` - Get pack details



POST `/api/v1/models/custom` - Save custom models



POST `/api/v1/models/set-model` - Set plan model



GET `/api/v1/providers` - List providers

---

## Phase 3: Context Management System (2-3 weeks)

### 3.1 Database Schema for Context



Migrations for context tables:



contexts (files, URLs, images, notes)



id, plan\_id, type, path, content, size



context\_versions (history)



id, context\_id, version, action, change\_summary



project\_maps (tree-sitter output)



id, project\_id, language, content, generated\_at



Context types enum: File, Directory, URL, Image, Note, Piped

### 3.2 Project Map Generation



internal/project/map/ package



generator.go: Main generation logic



tree\_sitter.go: Tree-sitter integration for 30+ languages



cache.go: Cache generated maps



language\_support.go: Language detection + parser loading



Features:



Support 30+ languages (use existing tree-sitter bindings)



Generate syntax trees with function/class/type definitions



Estimate token counts per file



Support projects up to 20M+ tokens



Incremental updates on file changes

### 3.3 Context Loading Pipeline



Stage 1: Discovery



`discovery.go`: Generate or load project map



Cache invalidation on file changes



Stage 2: Selection (Architect Phase)



`architect.go`: Call architect model to select relevant files



Input: project map + user prompt



Output: list of relevant file paths



Stage 3: Filtering (Smart Context)



`smart_filter.go`: Filter context by task step



Track which files are needed for implementation step



Only load those files into context



Dynamically adjust window size



Stage 4: Caching (Prompt Cache)



`caching.go`: Apply cache control directives



Mark cacheable sections (especially project map)



Track cache hits/savings



Only for providers with `SupportsCacheControl`

### 3.4 Context Management API



`internal/server/handlers/context.go`

☐

`POST /api/v1/plans/:id/context` - Add context

☐

`GET /api/v1/plans/:id/context` - Get context

☐

`DELETE /api/v1/plans/:id/context/:contextId` - Remove context

☐

`POST /api/v1/plans/:id/project-map` - Generate map

☐

`GET /api/v1/plans/:id/project-map` - Get map

### 3.5 Auto-Context Management

☐

Detect file changes outside Plandex

☐

Auto-update context when configured

☐

Conversation summarization when needed

☐

Config flag: `auto-update-context` (bool)

---

## Phase 4: File Editing & Diff System (1-2 weeks)

### 4.1 Diff Generation

☐

`internal/diff/` package

☐

`generate.go`: Create diffs via `git diff`

☐

`parse.go`: Parse unified diff format into hunks

☐

`model.go`: Data structures for diffs

☐

Replacement tracking:

☐

UUID per replacement for matching proposed ↔ actual

☐

Old text + new text + line number

## 4.2 Structured Edit System

- ☐ internal/syntax/ (expand existing)
  - ☐ Tree-sitter based AST-aware edits
  - ☐ Language-specific handlers
  - ☐ Generic fallback patterns
  - ☐ Validation + syntax checking

- ☐ Fallback chain:
  - ☐ 1. Try structured edit
  - ☐ 2. Syntax validation & fix
  - ☐ 3. Whole-file builder role
  - ☐ 4. Alternative builder approach
  - ☐ 5. Manual user fix

## 4.3 Change Application

- ☐ internal/changes/ package
  - ☐ apply.go: Apply changes to files
  - ☐ validate.go: Syntax validation before apply
  - ☐ rollback.go: Revert changes on error
  - ☐ staging.go: Selective file application
- ☐ Transactional support:
  - ☐ Atomic application of related changes
  - ☐ Rollback on partial failure

## 4.4 Diff Review Handlers

- ☐ `internal/server/handlers/changes.go`
  - ☐ GET `/api/v1/plans/:id/diff` - Get pending diff
  - ☐ POST `/api/v1/plans/:id/apply` - Apply changes
  - ☐ POST `/api/v1/plans/:id/reject` - Reject files
  - ☐ GET `/api/v1/plans/:id/changes` - List pending

## 4.5 Browser Diff UI Integration

- ☐ Export diff data to JSON format
  - ☐ Side-by-side and line-by-line views
  - ☐ File selection UI
  - ☐ Integration with existing UI
- 

# Phase 5: Plan Management & Versioning (1-2 weeks)

## 5.1 Plan Data Model

- ☐ Database migrations:
  - ☐ `plans` table
    - ☐ `id, name, status, autonomy_config, model_settings`
    - ☐ `created_at, updated_at, active_branch`
  - ☐ `plan_versions` table (complete history)
    - ☐ `id, plan_id, version_num, action, created_at`
    - ☐ `delta_summary, metadata`

- ☐
  - plan\_branches table
    - ☐ id, plan\_id, name, base\_version\_id
- ☐
  - pending\_changes table
    - ☐ id, plan\_id, file\_path, replacements (JSON)

## 5.2 Plan CRUD Operations

- ☐
  - internal/task/plan/crud.go
    - ☐ Create plan
    - ☐ Get plan
    - ☐ List plans (current directory + subdirs)
    - ☐ Update plan config
    - ☐ Delete plan
    - ☐ Archive/unarchive plan

## 5.3 Plan Versioning

- ☐
  - internal/task/plan/versioning.go
    - ☐ Create version on every action
    - ☐ Store minimal metadata (action, timestamp, summary)
    - ☐ Full content in separate tables
    - ☐ View history: plandex log
    - ☐ Rewind to previous version

## 5.4 Plan Branching

- ☐

`internal/task/plan/branching.go`

☐

Create branch from any version

☐

Switch between branches

☐

Merge branch back to main

☐

Conflict detection on rewind

## 5.5 Conversation Management

☐

`internal/task/plan/conversation.go`

☐

Store messages with streaming data

☐

Auto-summarization when max tokens exceeded

☐

Export full conversation

☐

View conversation history

## 5.6 Plan Handlers

☐

`internal/server/handlers/plans.go`

☐

POST /api/v1/plans - Create

☐

GET /api/v1/plans/:id - Get

☐

GET /api/v1/plans - List

☐

PUT /api/v1/plans/:id - Update

☐

DELETE /api/v1/plans/:id - Delete

☐

POST /api/v1/plans/:id/archive - Archive

☐

GET /api/v1/plans/:id/versions - History

☐

POST /api/v1/plans/:id/rewind - Rewind

☐

- ☐ GET /api/v1/plans/:id/branches - List branches
  - ☐ POST /api/v1/plans/:id/branch - Create branch
- 

## Phase 6: Autonomy System (1-2 weeks)

### 6.1 Autonomy Config Model



Data structure with 5 presets + individual flags

```
type AutonomyLevel string
const (
    AutonomyNone    = "none"
    AutonomyBasic   = "basic"
    AutonomyPlus    = "plus"
    AutonomySemi    = "semi"
    AutonomyFull    = "full"
)

type AutonomyConfig struct {
    Level          AutonomyLevel
    AutoContinue   bool // plan continuation
    AutoBuild      bool // file diff building
    AutoApply      bool // change application
    AutoExec       bool // command execution
    AutoDebug      bool // error debugging
    AutoCommit     bool // git commits
    AutoLoadCtx    bool // context loading
    SmartContext   bool // per-step filtering
    AutoUpdateCtx  bool // detect file changes
}
```

### 6.2 Preset Configurations



Implement 5 preset levels in code



none: All false



basic: AutoContinue, AutoBuild true



plus: ^ + SmartContext, AutoCommit, AutoUpdateCtx

☐

semi: ^ + AutoLoadCtx (default)

☐

full: All true

## 6.3 Runtime Autonomy Checks

☐

Feature guards throughout codebase:

☐

Before auto-continuing: Check AutoContinue

☐

Before auto-building: Check AutoBuild

☐

Before auto-applying: Check AutoApply

☐

Before auto-executing: Check AutoExec

☐

Before auto-debugging: Check AutoDebug

☐

Before auto-committing: Check AutoCommit

## 6.4 Handlers

☐

internal/server/handlers/autonomy.go

☐

GET /api/v1/plans/:id/autonomy - Current config

☐

POST /api/v1/plans/:id/autonomy - Set level

☐

POST /api/v1/plans/:id/autonomy-flag - Set individual flag

## 6.5 Safety Features

☐

Warn when setting full auto mode

☐

Require manual confirmation for destructive operations

☐

Suggest checking git status before full auto

☐

Document safety considerations

---

## Phase 7: Execution & Debugging (1-2 weeks)

### 7.1 Command Execution

- ☐ `internal/task/execution/runner.go`
  - ☐ Execute shell commands
  - ☐ Capture stdout/stderr
  - ☐ Timeout handling
  - ☐ Return exit codes

### 7.2 Auto-Debugging

- ☐ `internal/task/debugging/` package
  - ☐ `terminal.go`: Parse terminal output for errors
  - ☐ `browser.go`: Screenshot on web app errors
  - ☐ `analyzer.go`: Feed errors back to model

### 7.3 Execution Handlers

- ☐ `internal/server/handlers/execution.go`
  - ☐ `POST /api/v1/plans/:id/exec` - Run command
  - ☐ `GET /api/v1/plans/:id/exec-status` - Status

### 7.4 Rollback on Failure

- ☐ Track execution success/failure
  - ☐ Auto-rollback changes if commands fail
  - ☐ Preserve git state for manual recovery
-

## Phase 8: Git Integration (1 week)

### 8.1 Git Operations

- ☐ `internal/git/` package
  - ☐ `status.go`: Check git status
  - ☐ `commit.go`: Auto-commit with AI-generated messages
  - ☐ `revert.go`: Revert commits on rewind
  - ☐ `branch.go`: Create/checkout branches

### 8.2 Commit Message Generation

- ☐ Call LLM to generate commit messages
- ☐ Use `commit-messages` role
- ☐ Summarize changes from pending updates

### 8.3 Config

- ☐ `auto-commit` flag in autonomy
  - ☐ Optional git integration (works without repo)
- 

## Phase 9: Testing & Validation (Throughout, 2-3 weeks)

### 9.1 Unit Tests

- ☐ Provider integration tests
- ☐ Model resolution tests
- ☐ Context management tests

☐

Diff generation tests

☐

Plan CRUD tests

☐

Autonomy config tests

## 9.2 Integration Tests

☐

Full workflow: Create plan → Load context → Generate → Review → Apply

☐

Fallback chain tests

☐

Error recovery tests

☐

Rollback tests

## 9.3 E2E Tests

☐

CLI workflow tests

☐

API endpoint tests

☐

Database migration tests

## 9.4 Load/Performance Tests

☐

Context size scaling (up to 20M tokens)

☐

Large project map generation

☐

Multiple concurrent plans

---

# Phase 10: Documentation & Finalization (Throughout, 1 week)

## 10.1 Code Documentation

☐

- ☐ Godoc comments for all public functions
- ☐ Architecture docs for each package
- ☐ Provider-specific implementation notes

## 10.2 User Documentation

- ☐ Model configuration guide
- ☐ Context management guide
- ☐ Autonomy levels explanation
- ☐ Troubleshooting guide

## 10.3 Developer Guide

- ☐ Adding new providers
- ☐ Adding new models
- ☐ Extending context system
- ☐ Custom autonomy configs

## 10.4 Migration Guide

- ☐ From HelixCode v1 features
- ☐ Backward compatibility notes

---

# Implementation Notes

## Critical Decisions

1. **LiteLLM vs Direct:** LiteLLM recommended for abstraction but optional
2. **Database:** PostgreSQL with 50+ migration files (extensive)
3. **Language Support:** Start with top 10 languages, expand later
4. **Fallback Strategy:** Always have graceful degradation path

5. **Context Caching:** Optimize for token usage, track savings

## Reusable Components from HelixCode

- Database layer (PostgreSQL ready)
- REST API framework (Gin)
- Authentication system (JWT)
- Worker pool management (if applicable)
- CLI framework

## External Dependencies to Add

- `smacker/go-tree-sitter` (code analysis)
- `go-openai` (for direct OpenAI if not using LiteLLM)
- `aws-sdk-go` (for Bedrock support)
- `google-cloud-aiplatform` (for Vertex)

## Timeline Overview

- **Week 1-3:** LLM providers + model config
- **Week 4-6:** Context management + storage
- **Week 7-9:** Execution + versioning
- **Week 10-12:** Polish + integration + tests

**Estimated MVP:** 8-12 weeks of focused development

---

## Success Criteria

- ☐ All 12 providers working with fallback
- ☐ 25+ built-in models configured
- ☐ 16 model packs available
- ☐ Context system handles 20M+ token projects
- ☐ 5 autonomy levels configurable
- ☐ Full plan versioning with rewind
- ☐ Diff review sandbox working
- ☐

- ☐ Git integration functional
- ☐ 80%+ code coverage
- ☐ Documentation complete
- ☐ E2E tests passing